In [10]:
```
# %pip install pandas numpy matplotlib seaborn scikit-learn scipy openpyxl p
```

In [28]:
```
#%pip install StandardScalar
```

```
ERROR: Could not find a version that satisfies the requirement StandardScala
r (from versions: none)
ERROR: No matching distribution found for StandardScalar
Note: you may need to restart the kernel to use updated packages.
```

In [ ]:
```
# Classify the email using the binary classification method. Email Spam
# detection has two states: a) Normal State – Not Spam, b) Abnormal State –
# Spam. Use K-Nearest Neighbors and Support Vector Machine for classificatio
# Analyze their performance. Dataset link: The emails.csv dataset on the Kag
# ttps://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-
```

In [1]:
```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline

from sklearn.metrics import accuracy_score, confusion_matrix, classificatior
```

In [2]:
```
data = pd.read_csv("emails.csv")
```

In [3]:
```
print(data.head())
print(data.isnull().sum())
```

```
  Email No.  the  to  ect  and  for  of    a  you  hou  ...  connevey  jay
\
0   Email 1    0   0    1    0    0   0    2    0    0  ...         0    0
1   Email 2    8  13   24    6    6   2  102    1   27  ...         0    0
2   Email 3    0   0    1    0    0   0    8    0    0  ...         0    0
3   Email 4    0   5   22    0    5   1   51    2   10  ...         0    0
4   Email 5    7   6   17    1    5   2   57    0    9  ...         0    0

   valued  lay  infrastructure  military  allowing  ff  dry  Prediction
0       0    0               0         0         0   0    0           0
1       0    0               0         0         0   1    0           0
2       0    0               0         0         0   0    0           0
3       0    0               0         0         0   0    0           0
4       0    0               0         0         0   1    0           0

[5 rows x 3002 columns]
Email No.     0
the           0
to            0
ect           0
and           0
             ..
military      0
allowing      0
ff            0
dry           0
Prediction    0
Length: 3002, dtype: int64
```

In [4]:
```
X = data.drop(columns=['Email No.', 'Prediction'])
y = data['Prediction']
```

In [5]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

In [6]:
```
# 5. KNN Model
knn = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)


knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

In [7]:
```
# 6. SVM Model
svm = make_pipeline(
    StandardScaler(),
    LinearSVC(max_iter=10000, random_state=42)
)

svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)
```

In [8]:
```
# 7. KNN Performance
print("\nKNN RESULTS")
print("Accuracy:", accuracy_score(y_test, knn_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, knn_pred))
print(classification_report(y_test, knn_pred))
```

```
KNN RESULTS
Accuracy: 0.8253865979381443
Confusion Matrix:
[[846 251]
 [ 20 435]]
              precision    recall  f1-score   support

           0       0.98      0.77      0.86      1097
           1       0.63      0.96      0.76       455

    accuracy                           0.83      1552
   macro avg       0.81      0.86      0.81      1552
weighted avg       0.88      0.83      0.83      1552
```

In [9]:
```
# 8. SVM Performance
print("\nSVM RESULTS")
print("Accuracy:", accuracy_score(y_test, svm_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, svm_pred))
print(classification_report(y_test, svm_pred))
```

```
SVM RESULTS
Accuracy: 0.9317010309278351
Confusion Matrix:
[[1003   94]
 [  12  443]]
              precision    recall  f1-score   support

           0       0.99      0.91      0.95      1097
           1       0.82      0.97      0.89       455

    accuracy                           0.93      1552
   macro avg       0.91      0.94      0.92      1552
weighted avg       0.94      0.93      0.93      1552
```

In [10]:
```
# 9. Compare models
print("\nMODEL COMPARISON")
print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
```

```
MODEL COMPARISON
KNN Accuracy: 0.8253865979381443
SVM Accuracy: 0.9317010309278351
```

In [ ]: